



**COLÉGIO
DE GAIA**

Curso com Plano Próprio
Informática e Tecnologias Multimédia

Prova de aptidão profissional

12.º ano de escolaridade

Relatório

R2D2

Aluno
Artur Vicente da Cruz

Trabalho realizado sob a orientação da professora
Maria José Costa

Vila Nova de Gaia, junho de 2025

Dedicatória

*O passado é o que define o presente,
E o futuro é o motivo do presente*

Agradecimentos

Gostaria de agradecer a todos aqueles que, de alguma forma, gastaram parte do seu tempo para me ouvir e ajudar. Em especial à minha irmã Andreia que foi quem deu a identidade visual a este projeto. Aos professores do Colégio de Gaia que sempre se preocuparam e deram o seu melhor para eu ser o melhor possível. Aos meus colegas, pelo suporte que foram ao longo destes três anos, foi através deles que pude aprender, ensinar e aproveitar o ambiente escolar. E por último gostaria de agradecer a todos aqueles que estiveram lá, porque sem eles eu não seria quem sou hoje, tenham sido boas ou más influências.

Resumo

O presente relatório descreve o desenvolvimento da “R2D2”, uma linguagem de programação funcional criada de raiz. O principal objetivo foi conceber e implementar a sua sintaxe e a geração de código *JavaScript*. O projeto incluiu também a criação de uma interface de linha de comandos e um *website* para documentação. O relatório abrange todas as fases do processo, desde o conceito e planeamento até à implementação técnica, marcando o meu primeiro contacto com o desenvolvimento de linguagens de programação.

Palavras-chave: linguagem de programação, funcional, sintaxe, *JavaScript*

Abstract

This report presents the development of “R2D2,” a functional programming language created from scratch. The main objective was to design and implement its syntax and *JavaScript* code generation. The project also included the development of a command-line interface and a *website* for documentation. The report covers all phases of the process, from concept and planning to technical implementation, marking my first experience with programming language development.

Keywords: programming language, functional, syntax, JavaScript

Índice geral

Agradecimentos	v
Resumo	vii
Abstract	ix
Lista de abreviaturas	15
1 Introdução	16
2 Historial da instituição de ensino	17
2.1 Os níveis de ensino	18
2.2 O curso de Informática e Tecnologias Multimédia	19
3 Prova de Aptidão Profissional	22
3.1 Título do Projeto	22
3.2 Tema do Projeto	22
3.3 Motivação para o projeto	22
3.4 Contextualização no mercado (análise SWOT/FOFA)	23
3.4.1 Pontos fortes (<i>Strengths</i>)	23
3.4.2 Pontos fracos (<i>Weaknesses</i>)	24
3.4.3 Oportunidades (<i>Opportunities</i>)	24
3.4.4 Ameaças (<i>Threats</i>)	24
3.4.5 Comparação da R2D2 com Linguagens Estabelecidas	25
3.4.6 Conclusão do estudo de mercado	25
3.5 Ferramentas	25
3.5.1 Abordagem histórica	26
3.5.2 Justificação da utilização das ferramentas	29
4 Desenvolvimento do projeto	34
4.1 Calendarização inicial	34
4.1.1 Outubro	34
4.1.2 Novembro	34
4.1.3 Dezembro	34
4.1.4 Janeiro	35
4.1.5 Fevereiro	35
4.1.6 Março	35
4.1.7 Abril	35
4.1.8 Maio	36
4.1.9 Junho	36
4.2 Pesquisa	36
4.2.1 Setembro a novembro	36
4.2.2 Dezembro a janeiro	36
4.2.3 Fevereiro	37
4.2.4 Março a junho	37
4.3 Desenvolvimento	37

4.3.1	Desenvolvimento da interface de linha de comandos	38
4.3.2	Desenvolvimento da gramática	38
4.3.3	Desenvolvimento da linguagem de programação	38
4.3.4	Desenvolvimento do <i>website</i>	39
4.3.5	Desenvolvimento do Logótipo	39
4.3.6	Testes e feedback	41
5	Resultado final.....	42
5.1	Sintaxe da linguagem	42
5.1.1	Comentários	42
5.1.2	Use	42
5.1.3	Interfaces	43
5.1.4	Módulos	44
5.1.5	Funções	44
5.1.6	Variáveis e constantes	45
5.1.7	Instruções condicionais	46
5.1.8	Loops	47
5.1.9	For	47
5.1.10	While	48
5.1.11	Loop	48
5.2	Interface de Linha de Comandos.....	48
5.2.1	Version.....	48
5.2.2	Help	49
5.2.3	Run	50
5.2.4	Build.....	51
5.2.5	Js	51
5.3	<i>Website</i>	52
6	Melhorias futuras	55
6.1	Executar comandos através do menu de ajuda da CLI	55
6.2	Página <i>web</i> de experimentação da linguagem.....	55
6.3	Tipagem estática	55
6.4	Deteção automática de funções JavaScript	56
6.5	Criação de uma LSP.....	56
6.6	Outros planos	56
	Conclusão	57
	Bibliografia	58
	Glossário	59

Índice de Tabelas

Tabela 1 Comparação da R2D2 no contexto FOFA	25
--	----

Índice de Gráficos

Gráfico 1 Gráfico da facilidade mediante o tempo decorrido do desenvolvimento do projeto.....	39
---	----

Índice de Figuras

Figura 1 Programa do curso de ITM	20
Figura 2 Logótipo do <i>Arch Linux</i>	26
Figura 3 Logótipo do <i>Neovim</i>	26
Figura 4 Mascote do <i>Go</i>	27
Figura 5 Logótipo do <i>Word</i>	27
Figura 6 Logótipo do <i>Github</i>	27
Figura 7 Logótipo do <i>Deno</i>	28
Figura 8 Página de rabiscos do logótipo.....	40
Figura 9 Exemplo de comentários em R2D2	42
Figura 10 Comando <i>use</i> em R2D2	42
Figura 11 Declaração de interface em R2D2.....	43
Figura 12 Implementação da interface “Hello” no módulo “World” em R2D2.....	43
Figura 13 Exemplo de módulo em R2D2.....	44
Figura 14 Declaração de uma função simples em R2D2	44
Figura 15 Exemplo de declaração de uma função exportada em R2D2	45
Figura 16 Exemplo de declaração de uma pseudo-função em R2D2	45
Figura 17 Exemplo de uma chamada de função em R2D2	45
Figura 18 Exemplo da declaração de uma variável em R2D2	45
Figura 19 Exemplo de definição de uma constante em R2D2	45
Figura 20 Exemplo de implementação do <i>if</i> e do <i>else</i> em R2D2.....	46
Figura 21 Exemplo de implementação do <i>switch</i> em R2D2	46
Figura 22 Sintaxe do <i>break</i> em R2D2.....	47
Figura 23 Sintaxe do <i>continue</i> em R2D2.....	47
Figura 24 Exemplos de diferentes maneiras de fazer um <i>for</i>	47
Figura 25 Exemplo de como fazer um <i>while</i> em R2D2.....	48
Figura 26 Exemplo da implementação do <i>loop</i>	48
Figura 27 Exemplo da execução do comando <i>version</i>	48

Figura 28 Menu inicial do comando <i>help</i>	49
Figura 29 Página de detalhes de um comando	49
Figura 30 Comando <i>help static</i>	50
Figura 31 Exemplo de execução do comando <i>run</i> em R2D2	50
Figura 32 Exemplo da execução do comando <i>build</i> da CLI da R2D2	51
Figura 33 Exemplo da execução do comando <i>build</i> com o argumento “-o” em R2D2	51
Figura 34 Exemplo da execução do comando <i>js</i> da linha de comandos da R2D2	51
Figura 35 Exemplo da execução do comando <i>js</i> da CLI da R2D2 com o argumento “-o”	51
Figura 36 Página inicial.....	53
Figura 37 Página da documentação	54

Lista de abreviaturas

ANTLR	<i>ANother Tool for Language Recognition</i>
AST	<i>Abstract Syntax Tree</i>
AUR	<i>Arch User Repository</i>
CLI	<i>Command Line Interface</i>
CSS	<i>Cascading Style Sheets</i>
FCT	Formação em Contexto de Trabalho
FOFA	Forças, Oportunidades, Fraquezas e Ameaças (equivalente a SWOT)
GUI	<i>Graphical User Interface</i>
HTML	<i>HyperText Markup Language</i>
IR	<i>Intermediate Representation</i>
JS	<i>JavaScript</i>
KISS	<i>Keep It Simple, Stupid</i>
LLVM	<i>Low Level Virtual Machine</i>
PHP	<i>Hypertext Preprocessor</i>
SO	<i>Sistema Operativo</i>
SWOT	<i>Strengths, Weaknesses, Opportunities, Threats</i>
TUI	<i>Text User Interface</i>
UI	<i>User Interface</i>

1 Introdução

Neste trabalho, propõe-se o desenvolvimento de uma linguagem de programação própria, intitulada R2D2, criada com o objetivo de aprofundar conhecimentos nas áreas da programação, da criação de linguagens formais e da transpilação para *JavaScript*. Este projeto cruza componentes técnicas e criativas, desde o desenvolvimento da estética do *website* até à implementação da linguagem propriamente dita com ferramentas como ANTLR (*ANother Tool for Language Recognition*) e Go.

A escolha deste tema surgiu do meu interesse pessoal pela construção de ferramentas de baixo nível, especialmente no contexto do desenvolvimento. Apesar de temas como compiladores e linguagens formais não serem referidos ao longo do curso, senti necessidade de os explorar de forma prática e autónoma, aplicando conhecimentos de várias disciplinas num projeto ambicioso e desafiante.

Este relatório está dividido em três partes: na primeira secção, é apresentado o enquadramento da Prova de Aptidão Profissional (PAP), tendo em conta o contexto da instituição de ensino e os objetivos pedagógicos definidos. A segunda parte descreve o processo de conceção e desenvolvimento da linguagem R2D2, abordando aspetos como o desenvolvimento do *website* e o transpilador para *JavaScript*. Por fim, a última secção apresenta o resultado final da linguagem, limitações encontradas e possíveis melhorias e funcionalidades futuras.

Ao longo do relatório haverá momentos em que será utilizada a primeira pessoa do singular, de maneira a aprofundar os temas abordados e de modo a explicar melhor a mentalidade por trás de certas decisões.

2 Historial da instituição de ensino

Em 1933, na “Quinta do Trancoso”, nascia o Colégio Externato de Gaia por iniciativa do então Bispo do Porto, D. António Meireles. Localizada na freguesia de Mafamude, Vila Nova de Gaia, a referida Quinta foi doada à Diocese do Porto por D. Maria Margarida Guimarães e Silva.

Em 1934 foi concedido alvará de funcionamento ao Colégio Externato de Gaia – hoje Colégio de Gaia – sendo nomeado para diretor o Sr. Pe. Nédio de Sousa.

Nas décadas de 1960 e 1970 assistiu-se a uma significativa expansão das instalações, transformando o Colégio de Gaia no maior Colégio do país ao nível de instalações escolares e desportivas e de espaços exteriores, situação esta que se mantém até aos dias de hoje.

Atualmente e sob a direção do Sr. Pe. António Manuel Barbosa Ferreira, o Colégio de Gaia possui um Projeto Educativo próprio, de forma a servir uma população escolar com cerca de 1500 alunos que frequenta níveis de ensino desde os 3 anos até ao 12º ano de escolaridade.

Tendo como entidade titular a Diocese do Porto e sendo uma instituição de ensino católica, o Colégio de Gaia assegura uma educação sólida e assente em valores que fomentam um diálogo persistente e continuado com os pais e encarregados de educação centrando as nossas preocupações nos alunos e na sua formação e educação constante.

2.1 Os níveis de ensino

O Colégio de Gaia predispõe de três níveis de ensino, a saber:

- **Ensino Pré-escolar:** disponibiliza a entrada de alunos dos 3 aos 5 anos de idade e visa ajudar no desenvolvimento da individualidade da criança, promovendo a melhoria de aspetos pessoais, sociais e emocionais através de atividades estimulantes à sua evolução em aspetos cognitivos, expressivos e motores, incentivando-a a explorar os seus arredores de forma que esta se sinta segura e se encontre num ambiente saudável e cuidado.
- **Ensino Básico:** é composto pelo 1º Ciclo (compreendido entre o 1º e o 4º ano de escolaridade), pelo 2º Ciclo (compreendido entre o 5º e o 6º ano de escolaridade) e o 3º Ciclo (compreendido entre o 7º, 8º e o 9º ano de escolaridade), sendo os seus objetivos os seguintes:

1º Ciclo: trata-se da introdução do aluno ao ambiente escolar a que se irá conformar durante os restantes anos da sua escolaridade. Pretende-se que o aluno consiga desenvolver as suas capacidades e aptidões em áreas mais específicas através da atribuição de disciplinas. Esta fase é crucial para a formação de uma adaptabilidade do indivíduo aos métodos de estudo e restantes atividades e elementos em espaço escolar e em sociedade.

2º Ciclo: serve como um intermediário da passagem do aluno a uma fase de dificuldade acrescida. A carga horária é dividida por novas disciplinas, permitindo que o aluno explore novos setores e temas.

3º Ciclo: nesta fase, o foco principal em relação ao aluno é a sua futura entrada no secundário. Novas disciplinas são novamente adicionadas à carga horária, exigindo, agora, um maior esforço para poder assegurar um melhor futuro ao educando.

- **Ensino Secundário:** nesta instituição, são ministrados 13 cursos com planos próprios aprovados pela Portaria n.º 272/2019, de 27 de agosto. Estes conferem uma dupla certificação, uma vez que, para além de o aluno poder prosseguir os seus estudos para o ensino superior, pode integrar-se no mercado de trabalho, já que o curso confere ao aluno o nível 4 de qualificação profissional, nos termos do anexo III da Portaria n.º 782/2009, de 23 de julho, que regula o Quadro Nacional de Qualificações (QNQ).

Abaixo encontram-se os nomes das respetivas áreas que o aluno pode escolher:

- DP-AE: Desenhador de Projetos – Arquitetura e Engenharia;
- AGD: Animação e Gestão Desportiva;
- EIA: Eletrónica Industrial e Automação;
- ETC: Eletrónica, Telecomunicações e Computadores;
- MDI: Mecânica e Design Industrial;
- AM: Administração e Marketing;
- CGE: Contabilidade e Gestão Empresarial;
- CM: Comunicação Multimédia;
- ITM: Informática e Tecnologias Multimédia;
- TSI: Tecnologias e Sistemas de Informação;
- AQB: Análises Químico-Biológicas;
- TdS: Tecnologias da Saúde;
- TSA: Tecnologias e Segurança Alimentar.

2.2 O curso de Informática e Tecnologias Multimédia

O curso com Plano Próprio de Informática e Tecnologias Multimédia constitui uma oferta diferenciadora, que agrega as vertentes informática, multimédia, gestão de redes e bases de dados, em conformidade com os princípios, visão, valores e áreas de competências do Perfil dos Alunos à Saída da Escolaridade Obrigatória. A estruturação do plano curricular permite aos alunos orientarem os seus percursos académicos e profissionais para áreas diversas, como a programação, a tecnologia *Web* e dispositivos móveis, as bases de dados, a gestão de Infraestrutura informáticas (redes e segurança), e os jogos digitais.

Este curso habilita os alunos a prosseguirem estudos em cursos superiores, tais como: Engenharia Informática; Engenharia de Sistemas de Informação; Engenharia Eletrotécnica e de Computadores; Engenharia de Sistemas; Engenharia e Desenvolvimento de Jogos Digitais; Informática de Gestão; Informática, Redes e Multimédia; Tecnologias de Informação e Multimédia; Design de Jogos Digitais; além de outros cursos das áreas das Tecnologias e das Ciências; o aluno pode, ainda, ingressar num Curso Técnico Superior Profissional (CTeSP) ou num Curso de Especialização Tecnológica (CET).

Relativamente às saídas profissionais, os alunos podem exercer atividades em empresas especializadas na conceção, desenvolvimento e instalação de *software*, aplicações para dispositivos móveis e sistemas operativos; departamentos de informática; empresas de comercialização e/ou manutenção de programas e equipamentos informáticos; empresas de instalação, reparação e manutenção de redes informáticas; empresas de desenvolvimento de produtos multimédia, outras empresas ou entidades do setor público ou privado.

Abaixo, podem ser verificadas as disciplinas e respetiva carga horária relativa ao curso:

Plano de Estudos

(conforme Portaria n.º 272/2019, de 27 de agosto)

		Carga Horária Semanal (minutos)			Carga Horária (horas)	
Disciplinas		10.º Ano	11.º Ano	12.º Ano	Ciclo de Formação (a)	
Componente de Formação Geral						
Português	Cidadania e Desenvolvimento (b)	180	180	240	339	
Língua Estrangeira I, II ou III (c)		180	120	–	173	
Filosofia		120	180	–	172	
Educação Física		120	120	–	138	
Subtotal		600	600	240	822	
Componente de Formação Científica						
Matemática A		240	240	360	474	
Física e Química A		240	360	–	344	
Subtotal		480	600	360	818	
Componente de Formação Tecnológica						
Moral, Ética e Deontologia	60	60	60	102		
Aplicações Informáticas	120	–	–	70		
Fundamentos e Arquitetura de Computadores	180	120	–	173		
Técnicas de Programação	300	180	–	277		
Implementação e Exploração de Bases de Dados	–	180	120	168		
Programação Internet	–	–	300	165		
Tecnologias e Desenvolvimento Multimédia	–	–	180	103		
Projeto Tecnológico	–	60	60	67		
Subtotal	660	600	720	1 125		
Formação em Contexto de Trabalho (e)	–	105	295	400		
Educação Moral e Religiosa (d) (e)	(27)	(27)	(27)	(81)		
Total	1 740	1 800	1 320	3 165		

(a) Carga horária do ciclo de formação a gerir pela escola, no âmbito da sua autonomia pedagógica.

(b) Componente desenvolvida com o contributo de disciplinas e componentes de formação.

(c) O(A) aluno(a), se tiver estudado apenas uma língua estrangeira no ensino básico (LE I), iniciará obrigatoriamente uma segunda língua estrangeira no ensino secundário (LE II). No caso de o(a) aluno(a) iniciar uma língua, tomando em conta as disponibilidades do estabelecimento de ensino, poderá cumulativamente dar continuidade à Língua Estrangeira I como disciplina facultativa, com aceitação expressa do acréscimo de carga horária. Aos alunos oriundos de sistemas educativos estrangeiros, aplica-se o disposto no artigo 11.º.

(d) Disciplina de frequência facultativa.

(e) Carga horária anual.

Figura 1 Programa do curso de ITM

Como verificado acima, o curso predispõe de 8 disciplinas técnicas únicas ao mesmo, a saber:

- **Moral, Ética e Deontologia:** esta disciplina visa ensinar o educando sobre a conduta moral e ética para o seu desenvolvimento como pessoa e futuro membro trabalhador da sociedade;
- **Aplicações Informáticas:** ensina o aluno a realizar operações com ferramentas de produtividade, processamento e edição de texto, a programar com folhas de cálculo, desenvolver apresentações gráficas, entre outras;

- **Fundamentos e Arquitetura de Computadores:** trata-se da aprendizagem sobre a instalação, manutenção e reparação de equipamentos informáticos, instalação e configuração de dispositivos internos e externos no computador, redes locais e serviços adicionais de rede, assim como a monitorização e gestão dos níveis de disponibilidade e desempenho da comunicação entre os mesmos, introduzindo modificações e operações de acerto;
- **Técnicas de Programação:** introdução à programação através de pseudocódigo, seguidamente da aprendizagem de duas linguagens de programação (*Visual Basic* e *C#*, respetivamente), procedendo, seguidamente, a testes de validação, tendo como objetivo levar o aluno a operar com base nos fundamentos gerais de programação;
- **Implementação e Exploração de Bases de Dados:** esta disciplina visa ensinar o indivíduo a instalar e configurar sistemas de gestão de bases de dados, a operar e administrar bases de dados e a preparar o tratamento dos dados, com vista a garantir um funcionamento fiável e flexível;
- **Programação Internet:** introdução à programação na *web* através da introdução de novas linguagens de programação específicas (*CSS*, *PHP*, *HTML* e *JavaScript*); pretende-se que o aluno consiga aprender a instalar e configurar servidores de *web*, e-mail e acesso à internet, criar páginas para a *web* e assegurar a segurança de redes;
- **Tecnologias e Desenvolvimento Multimédia:** introdução à gestão de projetos, desenvolvimento de jogos digitais e manipulação de vídeo e imagem;
- **Projeto Tecnológico:** formação para a aplicação dos conceitos com propósito de preparação para o mercado de trabalho.

Para além destas disciplinas, no 12º ano de secundário, o aluno terá de participar na FCT (Formação em Contexto de Trabalho), realizando estágio curricular em determinadas empresas com o objetivo de melhorar as suas capacidades de adaptabilidade a novos métodos de trabalho e flexibilidade para futura exercício no mercado de trabalho, assim como o desenvolvimento da sua própria pessoa, valores, capacidades e aptidões.

Como elemento final de avaliação, o aluno terá, ainda, de realizar um projeto final prático onde terá de pôr à prova todos os conhecimentos adquiridos no curso que frequentou seguindo o tema do projeto escolhido.

Tanto a FCT como a PAP terão de proporcionar tarefas ao aluno alusivas à matéria lecionada no curso através das disciplinas técnicas.

3 Prova de Aptidão Profissional

A Prova de Aptidão Profissional (ou PAP) consiste na execução e apresentação de um projeto final prático no qual o aluno pretende demonstrar as competências e conhecimentos que desenvolveu e adquiriu ao longo da formação proposta pelo curso que frequentou, incidindo sobre um tema específico.

Segundo o preceituado na Portaria 74A/2013 de 15 de fevereiro (artigo 17º, nº3), a concretização do projeto compreende quatro momentos essenciais: conceção, desenvolvimento, autoavaliação e elaboração do relatório final.

3.1 Título do Projeto

O nome da minha Prova de Aptidão Profissional (PAP) é R2D2, uma referência a um *droid* do universo de *Star Wars*, uma série filmes da qual sou um grande apreciador. Para além disso, a pronúncia em inglês deste conjunto de caracteres lê-se “are-tu-di-tu”, o que remete de forma lúdica à fonética do meu nome no diminutivo, Arturito.

3.2 Tema do Projeto

O tema do meu projeto consiste no desenvolvimento de uma linguagem de programação. A linguagem incorpora o conceito de “pseudo-funções”, que são funções que apenas podem invocar outras funções, promovendo uma organização mais explícita e estruturada do código. O objetivo é proporcionar uma maneira clara e intuitiva de organizar o código, facilitando a compreensão e a manutenção do sistema de informação.

3.3 Motivação para o projeto

Desde o 11.º ano, desenvolvi um grande interesse pela programação de baixo nível, através de linguagens como C. Ao chegar ao 12.º ano, tinha a certeza de que a minha Prova de Aptidão Profissional (PAP) seria um de três projetos:

1. Uma linguagem de programação
2. Um sistema operativo
3. Um programa desenvolvido em *assembly*

Após uma análise cuidadosa, optei por desenvolver uma linguagem de programação, por várias razões.

Em primeiro lugar, a criação de uma linguagem de programação seria uma expressão mais evidente da minha personalidade e visão.

Em segundo lugar, a implementação de um sistema operativo revelava-se talvez demasiado complexa, e os resultados obtidos poderiam ser limitados, levando-me a recear que o meu trabalho fosse menosprezado.

Em terceiro lugar, o desenvolvimento de um programa em *assembly* estaria automaticamente restrito pela arquitetura do meu computador, e eu não acho que faria um programa que tivesse alguma relevância, algo que

tento sempre evitar. Além disso, a implementação de várias funcionalidades consideradas comuns na programação aumentaria a incerteza quanto à qualidade do resultado final.

Dito isto, escolhi desenvolver uma linguagem de programação não por acreditar que seria a melhor linguagem de programação de sempre, mas porque, é um tema que genuinamente me interessa. Desde que comecei a programar, dediquei-me a conhecer e aprender diferentes linguagens de programação e sempre desejei ter a minha própria interpretação sobre como uma linguagem de programação deve ser.

Em quarto lugar, este tema representa um verdadeiro desafio para mim; já que eu achava que, se optasse por um projeto com o qual já tenho familiaridade, como um *website*, teria maior facilidade, mas isso não proporcionaria o mesmo entusiasmo no desenvolvimento da PAP.

Por último, a PAP é uma oportunidade única para me motivar a desenvolver um projeto desta envergadura. Conhecendo-me bem, sei que, se tentasse desenvolver este projeto de forma independente, iria muito provavelmente desistir devido à carga de trabalho envolvida em algo que não me indicaria progresso de maneira instantânea. Por último, ao construir um programa desta natureza, de certa maneira, tenho algo que eu posso olhar e lembrar-me de que eu tenho capacidade de fazer coisas que não tenho tanta familiaridade ou conforto.

3.4 Contextualização no mercado (análise SWOT/FOFA)

A análise FOFA (ou SWOT, em inglês) é uma ferramenta estratégica amplamente utilizada para identificar os fatores internos e externos que podem impactar o sucesso de um projeto. Para a minha linguagem de programação, esta análise é fundamental para compreender as suas principais forças, fraquezas, oportunidades e ameaças no mercado atual.

3.4.1 Pontos fortes (*Strengths*)

- Introdução de pseudo-funções, um conceito inovador que pode atrair programadores interessados em modularidade.
- Compatibilidade com *JavaScript*, uma das linguagens mais populares no desenvolvimento *web*.
- Estrutura simples, o que facilita a adoção por parte de programadores.

3.4.2 Pontos fracos (*Weaknesses*)

- A linguagem ainda está em desenvolvimento, não tendo uma implementação funcional ou histórico de utilização.
- Desconhecimento total por parte da comunidade de programadores, dificultando a criação de uma base inicial de utilizadores.
- Falta de ferramentas e bibliotecas disponíveis numa fase inicial.

3.4.3 Oportunidades (*Opportunities*)

- O crescente uso de *JavaScript* em múltiplos contextos de desenvolvimento, abrindo possibilidades para a R2D2.
- Nichos específicos do mercado que valorizam modularidade e previsibilidade podem beneficiar da adoção da linguagem.
- Potencial para criar uma comunidade de código aberto que contribua para o amadurecimento da R2D2.

3.4.4 Ameaças (*Threats*)

- Forte concorrência de linguagens já consolidadas, como *Rust*, *TypeScript*, entre outras.
- Dificuldade em demonstrar o valor da R2D2 frente a soluções já estabelecidas.
- Risco de baixa adoção devido à ausência de um ecossistema inicial robusto.

3.4.5 Comparação da R2D2 com Linguagens Estabelecidas

Antes de concluir este estudo, é essencial comparar a R2D2 com linguagens consolidadas no mercado. Abaixo está uma tabela que realça as principais características de cada linguagem e como a R2D2 se posiciona nesse cenário:

Característica	R2D2	Rust	Typescript	AssemblyScript	Go
Pseudo-funções	Sim	Não	Não	Não	Não
Compilação para JS	Sim	Não	Sim	Sim	Não
Conhecimento atual	Nenhum	Extenso	Extenso	Médio	Extenso
Ecosistema	Inexistente	Amplio	Amplio	Médio	Amplio
Curva de aprendizagem	Moderada	Íngreme	Leve	Moderada	Leve

Tabela 1 Comparação da R2D2 no contexto FOFA

A partir desta tabela pode-se concluir que a R2D2 possui características únicas, como o conceito de pseudo-funções, que podem se tornar um diferencial competitivo, mas ainda há um longo caminho para alcançar o nível de robustez e maturidade de linguagens como *Rust*, *TypeScript* e *Go*. A curva de aprendizagem moderada e o foco na modularidade são pontos positivos, porém a falta de um ecossistema pode limitar sua adoção inicial.

3.4.6 Conclusão do estudo de mercado

A análise FOFA permitiu identificar os principais fatores que podem influenciar o sucesso da linguagem R2D2. Os pontos fortes mostram o potencial de inovação e aplicação da linguagem, enquanto os pontos fracos e as ameaças reforçam a necessidade de superar barreiras como a criação de um ecossistema inicial e a atração de uma base de utilizadores. Contudo, as oportunidades destacadas representam uma possibilidade única de evolução e afirmação da linguagem num mercado competitivo, particularmente ao explorar nichos que valorizem as suas características diferenciadoras.

3.5 Ferramentas

Neste capítulo, será apresentada a escolha das ferramentas utilizadas no desenvolvimento do projeto, assim como as ferramentas auxiliares para a elaboração do relatório final. Será feita uma breve abordagem à história de cada uma, bem como ao tempo dedicado à sua aprendizagem e compreensão. Serão ainda destacadas as funcionalidades de cada ferramenta e de que forma contribuem para facilitar o processo de desenvolvimento, organização e documentação do projeto.

3.5.1 Abordagem histórica

Neste subcapítulo, será feita uma breve análise histórica das principais ferramentas utilizadas no projeto, destacando a sua origem, evolução e impacto na área do desenvolvimento de *software*. Esta contextualização permite compreender melhor o papel de cada ferramenta.

3.5.1.1 Sistema operativo

Arch Linux foi criado em 2002 por Judd Vinet com o objetivo de ser uma distribuição simples, leve e personalizável. Baseado na filosofia KISS ("Keep It Simple, Stupid"), oferece um sistema minimalista e um modelo de atualizações contínuas (*rolling release*). Destaca-se pela flexibilidade e pela comunidade ativa, especialmente pela sua documentação, a *Arch Wiki*. Desde 2007, é mantido por uma equipa de desenvolvimento que continua a melhorar a distribuição, tornando-a popular entre utilizadores avançados.



Figura 2 Logótipo do Arch Linux

3.5.1.2 Editor de texto

O *Neovim* foi lançado em 2014 como um *fork* do editor "*Vim*", com o objetivo de modernizar o código, melhorar a extensibilidade e facilitar a manutenção. Mantendo a essência e a filosofia do *Vim*, o *Neovim* introduziu melhorias como suporte nativo para *plugins* assíncronos e uma arquitetura mais modular. Desde então, tem crescido em popularidade, especialmente entre programadores que procuram um editor leve, rápido e altamente customizável.



Figura 3 Logótipo do Neovim

3.5.1.3 Linguagem de programação

O Go, também conhecido como *Golang*, foi criado pelo *Google* em 2007 por Robert Griesemer, Rob Pike e Ken Thompson, e lançado publicamente em 2009 com o objetivo de ser uma linguagem de programação simples, eficiente e moderna, que facilitasse o desenvolvimento de *software* escalável e de alto desempenho. Go destaca-se pela sua sintaxe clara, suporte nativo a programação concorrente através de *goroutines* e canais, e pela combinação da facilidade de linguagens como *Python* com a velocidade e controlo oferecidos por linguagens mais próximas do *hardware* como o C. Desde o seu lançamento, tem vindo a ser amplamente adotado em áreas como desenvolvimento de servidores *web*, ferramentas de infraestrutura e aplicações em *cloud computing*, tornando-se uma das linguagens mais populares no desenvolvimento moderno.

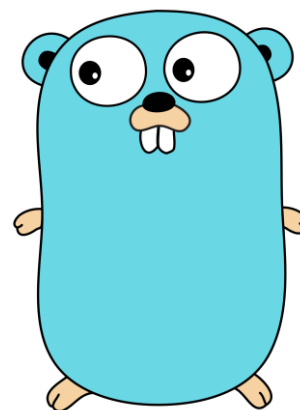


Figura 4 Mascote do Go

3.5.1.4 Word

O *Microsoft Word* é um programa de processamento de texto desenvolvido pela Microsoft, lançado inicialmente em 1983. Desde então, tem evoluído para se tornar uma das ferramentas mais populares e utilizadas mundialmente para criação, edição e formatação de documentos digitais. Com uma interface intuitiva e um conjunto abrangente de funcionalidades, o *Word* permite aos utilizadores produzir desde simples textos até documentos complexos com imagens, tabelas, gráficos e estilos variados. A sua integração com o pacote *Microsoft Office* e a constante atualização tornam-no uma ferramenta essencial tanto para utilizadores individuais como para ambientes profissionais e educativos.



Figura 5 Logótipo do Word

3.5.1.5 Versionamento do projeto e backups

O *GitHub*, lançado em 2008, é uma plataforma que permite armazenar, gerir e partilhar código usando o sistema de controlo de versões *Git*. Facilita a colaboração entre programadores através de funcionalidades como *pull requests* e *issues*, sendo uma das maiores comunidades de desenvolvimento de *software* do mundo. Em 2018, foi adquirido pela *Microsoft*, o que reforçou a sua integração com outras ferramentas de desenvolvimento.



Figura 6 Logótipo do GitHub

3.5.1.6 Deno

O *Deno* foi lançado em 2018 por Ryan Dahl, criador do *Node.js*, como uma versão modernizada para executar *JavaScript* e *TypeScript* fora do navegador. Desenvolvido para corrigir limitações e problemas de segurança do *Node.js*, o *Deno* inclui suporte nativo para *TypeScript*, um modelo de permissões seguro e uma arquitetura simplificada. Desde então, tem ganho destaque como uma alternativa leve e segura para execução de *scripts* do lado do servidor.



Figura 7 Logótipo do Deno

3.5.1.7 Outras ferramentas

Além das ferramentas mencionadas, utilizarei as seguintes tecnologias para a execução deste projeto:

ANTLR

Ferramenta poderosa para gerar analisadores sintáticos (*parsers*) a partir de gramáticas, facilitando o desenvolvimento de compiladores e interpretadores.

JavaScript

Linguagem de programação popular para *web*, usada para adicionar dinamismo e interatividade a páginas e aplicações *web*.

Templ

Motor de *templates* leve para gerar HTML (*HyperText Markup Language*) dinâmico, usado para separar lógica de apresentação em aplicações *web*.

Templ UI

Biblioteca associada ao *Templ* para construir interfaces gráficas simples e eficientes, facilitando a criação de componentes visuais.

Tailwind CSS

Framework de CSS (*Cascading Styling Sheets*) baseado em classes utilitárias que permite criar estilos personalizados de forma rápida e consistente.

Bubbles e Bubbletea

Bibliotecas em *Go* para construir interfaces interativas em linha de comandos, com suporte a componentes reutilizáveis e eventos.

3.5.2 Justificação da utilização das ferramentas

Neste subcapítulo, será apresentada a justificação para a escolha das ferramentas utilizadas ao longo do desenvolvimento do projeto. Cada ferramenta foi selecionada com base nas suas funcionalidades, adequação aos objetivos do projeto e na eficiência que proporcionou durante a sua implementação.

3.5.2.1 Sistema Operativo

Já há mais de um ano que utilizo sistemas baseados em *Linux* nas minhas máquinas pessoais e para este projeto não será diferente, de entre as várias distribuições optei pelo *Arch Linux*, principalmente pelo AUR (*Arch User Repository*), o gestor de pacotes com que ele vem, que de todos os sistemas *Linux* que experimentei, é o mais completo, então, sabendo que o projeto poderia envolver várias tecnologias diferentes desde o início estava bastante claro que utilizaria este SO (Sistema Operativo).

3.5.2.2 Editor de texto

Quanto ao editor de texto, como mencionado anteriormente, utilizarei o *Neovim* como editor de texto. De maneira semelhante ao Sistema Operativo, já utilizava este editor antes do projeto. Utilizar este editor em vez de outras opções mais populares como o *Visual Studio Code* deve-se a vários fatores.

Primeiramente, a integração diretamente com o *terminal*. Apesar de não parecer tão relevante, já que a maioria dos editores modernos contêm um terminal embutido, mas o *Neovim* funciona de maneira diferente, em vez de conter um *terminal*, ele está contido no *terminal*, esta característica torna-o um editor muito mais leve e rápido que a maioria dos editores de texto.

Em segundo lugar, a customização. A grande maioria dos editores de texto atualmente têm alguma forma de customização, e, a meu ver, o *Neovim* é o mais personalizável de todos, já que praticamente qualquer aspeto da aplicação pode ser facilmente alterado, removido ou até adicionado, o que permite que a experiência de utilização seja o mais confortável possível.

Em último lugar, mas não menos importante, as *vim-motions*. Elas consistem essencialmente num conjunto de *shortcuts* que permitem a navegação e edição de texto, não só mais fácil para quem as conhece, mas também, de certa maneira, mais divertido, já que torna a adição e edição de texto algo mais dinâmico, algo que vai para além de escrever palavras.

3.5.2.3 Linguagem de Programação

A utilização da linguagem de programação certamente estará nas decisões mais difíceis no que toca à definição das ferramentas que utilizaria no projeto. Quando decidi efetuar este projeto já tinha algumas linguagens em mente, entre elas: *Zig*, *Rust*, *C* e *Go*. No início estava a tender para o *Zig* e para o *Rust*, devido à popularidade que elas têm em projetos deste género.

Numa primeira fase de eliminação, retirei o *C* pelos vários problemas que vêm inerentes com a utilização da linguagem, como a gestão da memória, e a falta de funcionalidades com que ela vem.

Pouco depois acabei por decidi não utilizar *Rust* por ser uma linguagem que, na minha ótica, é muito restrita, o que não me agrada já que o meu objetivo é desenvolver código e não ter a certeza de que estou a ser o mais eficiente e seguro possível.

Na fase final, tendo como opções: o *Zig* e o *Go*. Estava mais inclinado para o *Zig* devido à sua sintaxe única e da qual aprecio. No entanto, após alguma pesquisa no tema da criação de linguagens de programação, percebi que não havia nenhuma fonte que falasse do tema para o *Zig*, e como ainda não sabia como implementar precisava de uma tecnologia que me desse alguma base para o desenvolvimento do projeto. Fazendo com que a escolha final fosse o *Go*, o que se tornou a escolha perfeita, já que mais tarde tive a oportunidade de desenvolver um programa com esta linguagem no âmbito da minha FCT (Formação em Contexto de Trabalho), de atender um workshop sobre o mesmo e, felizmente, ter suporte para o desenvolvimento com ANTLR.

3.5.2.4 Word

Quanto à decisão da utilização do *Word*, foi uma escolha bastante fácil, já que nas aulas aprendemos a utilizar esta ferramenta para o desenvolvimento de relatórios e documentos semelhantes, tornando-a a ferramenta mais óbvia para o desenvolvimento deste relatório.

3.5.2.5 Versionamento do projeto e backups

Quanto ao versionamento do projeto, a utilização da *GitHub* é uma decisão que me foi natural, já que já o tinha utilizado para projetos anteriores, incluindo no meu trabalho da FCT, e, de facto, a estrutura da *Git* facilita muito a criação de *commits* e o *rollback* para os mesmos se necessário. Para além disso, a integração de pacotes *Go* no *GitHub* incentivou também a escolha

3.5.2.6 Deno

O *Deno* certamente foi a escolha menos pensada do projeto, mas com certeza a mais acertada. Tendo como principais escolhas de *runtimes* de JS para a minha aplicação: *Node* e *Deno*. Decidi utilizar *Deno* apenas porque tinha a certeza que queria utilizar o *Node* e, sendo o *Deno* criado pelo criador do *Node* pensei algo como: “Se o criador do *Node* criou outra ferramenta que faz, basicamente a mesma coisa que o próprio *Node*, então devo utilizar essa ferramenta.”. Mais tarde vim a descobrir outro *runtime* chamado *Bun* que era feito no próprio Go, mas por ter já implementado o *Deno* não me dei ao trabalho de explorar a possibilidade de utilizá-lo, embora que o fator de ser feito em Go poderia permitir uma melhor integração com o *runtime*.

3.5.2.7 Outras ferramentas

ANTLR (ANOther Tool for Language Recognition)

A utilização do ANTLR, apesar de não ter sido planeada inicialmente, foi crucial para a realização deste projeto. Felizmente Go tinha suporte para esta ferramenta e graças a ela pude não só perceber melhor de que se trata o *lexer* e o *parser*, mas também ter uma estrutura base para o *visitor* da minha linguagem, o que foi muito bom de ter.

JavaScript

Inicialmente estava a planear em utilizar *WebAssembly* como o alvo de compilação da minha linguagem, já que de alguns vídeos que tinha visto, estavam sempre a dizer que o *WebAssembly* era o futuro e coisas do género, no entanto quando realmente “pus as mãos na massa” percebi que não iria conseguir fazê-lo, já que se tratava de uma linguagem de sintaxe muito específica que nunca tinha sequer experimentado utilizar antes. Por isso, sabendo que precisava de uma alternativa, fui a um servidor de *Discord* sobre linguagens de programação e mandei uma mensagem a explicar a minha situação, felizmente pouco tempo depois de mandá-la comecei a receber respostas e estavam todos a dizer-me para utilizar *JavaScript* e eu achei muito estranho porque se as pessoas já programam em *JavaScript* porque é que eu estaria a criar algo que compila para o próprio *JavaScript*?

À procura de mais respostas fui à um direto do “Kuviman” (algunha da pessoa na plataforma), e como ele próprio estava a programar a sua linguagem de programação “Kast” e fui ao seu chat perguntar-lhe se fazia sentido compilar a minha linguagem para *JavaScript*, e, aparentemente, a sua linguagem era compilada para *JavaScript* e ele explicou-me que não faz sentido estar a compilar a minha linguagem para coisas como *WebAssembly* já que ainda nem sequer tinha visto como é que se faz uma linguagem, fazer uma linguagem de programação e aprender a compilá-la para algo mais complexo são duas tarefas distintas e para alguém como eu que não tinha nenhuma experiência a fazer uma linguagem fazia mais que sentido utilizar *JavaScript*. E, como já tinha programado em *JavaScript*, simplesmente ouvi o conselho que me deram.

Templ

De facto, decidir a maneira como faria a página *web* foi a decisão que mais tempo demorou e que mais experimentações levou. Quando comecei pensei em fazer o site em *React*, mas ao começar o projeto percebi que estava a utilizar uma tecnologia muito complicada para alguma que era simples então fui à procura de uma solução mais prática e encontrei o *Starlight*, um tema de *Astro* destinado à criação de aplicações *web* de documentação. Cheguei a fazer um protótipo, mas achei muito complexo a mais porque, apesar de ter a conveniência de fazer as páginas com *markdown*, o processo de configurar o *site* para ter o aspeto que eu queria era muito forçado e complicado e ao perceber que estava a passar mais tempo a configurar o site do que, efetivamente, fazê-lo percebi que deveria tomar um caminho que sabia ser simples, o HTML, no início pareceu-me certo, mas rapidamente percebi que o HTML era talvez simplicidade a mais.

Depois de ter experimentado tudo e não ter feito nada, decidi que talvez deveria ter algo com a facilidade do HTML, mas ao mesmo tempo dar-me a potência de um *framework* como *React*, e quando vi o *framework* *Templ* tudo se alinhou, não só é um *framework* fácil e leve, como também tinha vários recursos e a potência de *frameworks* mais completos, e, para além de tudo isto, era em Go, ou seja, tinha as características todas que procurava e ainda tinha o conforto de uma linguagem que estava a usar e com que estava confortável.

Templ UI (User Interface)

De maneira a facilitar o desenvolvimento do *website*, utilizei o *Templ UI*, uma biblioteca de componentes muito semelhante a bibliotecas que têm se tornado populares nos últimos tempos, como o *ShadCN*, que basicamente tem como filosofia a ideia de que o código deve estar todo disponível para o programador editar e customizar para o seu próprio caso. E, como esta biblioteca tinha todos os componentes de que precisava e era destinada, como o nome indica, para o *framework* *Templ*, a escolha pareceu-me natural.

Tailwind CSS

Para estilizar as páginas do meu *website* optei por utilizar o *framework* de CSS *Tailwind CSS*, de projetos anteriores sabia que era uma opção confiável e que me permitiria customizar o *website* com uma rapidez e facilidade diferente do que utilizar CSS puro.

Bubbles e Bubbletea

Apesar de nunca ter utilizado componentes destinados para uma aplicação TUI (*Terminal User Interface*), sabia que existiam e quando comecei a desenvolver a CLI (*Command Line Interface*) do projeto fui à *GitHub* à procura e percebi que o *Bubbletea* era o repositório mais popular, quando decidi que ia utilizar esta biblioteca fiz um pouco mais de pesquisa e aparentemente havia outro *package*, que era um *wrapper* do próprio *Bubbletea* chamado *Bubbles* e, essencialmente, tudo o que fazia era melhorar o aspeto dos componentes e, depois de ver o aspeto dos componentes, ficou decidido que iria utilizá-los no projeto.

4 Desenvolvimento do projeto

Este capítulo será utilizado para listar e explorar o processo de desenvolvimento apresentando detalhes essenciais, desde calendarizações a tempos esperados e gastos em pesquisa, formação e produção, bem como as dificuldades que tive pelo meio.

4.1 Calendarização inicial

Neste subcapítulo serão abordados os meus planos de desenvolvimento do projeto no início do projeto. Vale a pena constar que o cumprimento rigoroso com os mesmos não foi realizado não só por causa da natureza do projeto e dos meus conhecimentos iniciais sobre o desenvolvimento do mesmo, mas também pelos planos contêm certos assuntos que ao longo do tempo demonstraram-se pouco importantes ou irrelevantes no contexto.

4.1.1 Outubro

Objetivo: Adquirir conhecimentos fundamentais.

- **Semanas 1-2:** Estudo de estruturas de dados importantes para o design de linguagens (árvores, tabelas de dispersão e listas ligadas).
- **Semanas 3-4:** Introdução aos compiladores (“The Dragon Book” ou “Engineering a Compiler”). Estudo de conceitos como análise léxica, análise sintática e geração de código.
- **Resultado Esperado:** Compreensão sólida das estruturas de dados essenciais e dos fundamentos dos compiladores.

4.1.2 Novembro

Objetivo: Definir a sintaxe e as funcionalidades principais da linguagem.

- **Semanas 1-2:** Estudo de gramáticas livres de contexto e árvores de sintaxe abstrata (ASTs).
- **Semanas 3-4:** Definição da sintaxe da linguagem e elaboração da gramática formal, incluindo elementos como variáveis, funções e estruturas condicionais.
- **Resultado Esperado:** Especificação da linguagem com regras sintáticas claras e esboço da gramática.

4.1.3 Dezembro

Objetivo: Criar um analisador léxico (*lexer*) para transformar o código fonte em *tokens*.

- **Semanas 1-2:** Implementação do *lexer* para identificar palavras-chave, identificadores, operadores e literais.
- **Semanas 3-4:** Refinamento e testes com exemplos de código.
- **Resultado Esperado:** *Lexer* funcional que converte o código fonte em *tokens* para o *parser*.

4.1.4 Janeiro

Objetivo: Construir um *parser* que gere a árvore de sintaxe abstrata (AST).

- **Semanas 1-2:** Estudo de algoritmos de *parsing* (como *parsing* descendente recursivo ou *Pratt parsing*).
- **Semanas 3-4:** Implementação do parser com base na gramática definida, gerando uma AST.
- **Resultado Esperado:** Parser funcional que transforma tokens em ASTs representando a lógica do código.

4.1.5 Fevereiro

Objetivo: Implementar a análise semântica e gerar uma representação intermédia do código.

- **Semanas 1-2:** Verificações semânticas como tipagem, escopo e resolução de funções.
- **Semanas 3-4:** Geração de uma representação intermédia (IR) a partir da AST (por exemplo, em formato de código de três endereços ou compatível com LLVM).
- **Resultado Esperado:** AST validada semanticamente e IR pronta para a geração de código.

4.1.6 Março

Objetivo: Transformar a IR em código de baixo nível (como *Assembly* ou *WebAssembly*).

- **Semanas 1-2:** Estudo de conceitos básicos de *Assembly* ou *WebAssembly*.
- **Semanas 3-4:** Implementação de um gerador de código para o alvo pretendido, com testes em exemplos simples.
- **Resultado Esperado:** Geração de código de baixo nível a partir da IR, concretizando o compilador.

4.1.7 Abril

Objetivo: Refinar o compilador com otimizações e tratamento de erros robusto.

- **Semanas 1-2:** Implementação de otimizações básicas como eliminação de código morto.
- **Semanas 3-4:** Adição de mecanismos de tratamento de erros no lexer, parser e gerador de código.
- **Resultado Esperado:** Compilador mais eficiente, com tratamento de erros adequado e otimizações aplicadas.

4.1.8 Maio

Objetivo: Garantir a robustez do projeto e preparar a entrega final.

- **Semanas 1-2:** Testes completos com diversos exemplos de código e casos-limite.
- **Semanas 3-4:** Redação da documentação final (especificação da linguagem, guia de utilização e exemplos). Preparação dos materiais da apresentação.
- **Resultado Esperado:** Projeto finalizado, testado e documentado, pronto para ser apresentado.

4.1.9 Junho

Objetivo: Apresentar o projeto de forma clara, estruturada e confiante.

- Demonstração do processo de compilação, desde o código-fonte até ao código de baixo nível, com exemplos práticos e explicação dos componentes principais do compilador.

4.2 Pesquisa

A pesquisa é um componente crucial no desenvolvimento de qualquer projeto, especialmente quando se trata da criação de uma linguagem de programação. Durante este período, procurei adquirir uma compreensão sólida dos conceitos fundamentais e das melhores práticas associadas à construção de linguagens. A pesquisa não apenas me permitiu explorar as diversas abordagens e ferramentas disponíveis, mas também ajudou a moldar a visão e os objetivos do meu projeto. Nos tópicos a seguir, descrevo as etapas que segui ao longo deste processo de pesquisa.

4.2.1 Setembro a novembro

Nesta fase inicial, dediquei-me a assistir a diversos vídeos e a ler artigos sobre a criação de linguagens de programação de uma maneira mais superficial, com o objetivo de perceber se esta ideia realmente era factível. Este período foi fundamental para experimentar e sentir um pouco do que era o desenvolvimento deste género de projetos.

4.2.2 Dezembro a janeiro

Nesta parte do projeto, utilizei o meu tempo para fazer uma pesquisa mais profunda sobre o desenvolvimento de linguagens de programação e que tecnologias é que deveria utilizar, de maneira a poder ajudar-me a longo prazo, como foi o caso do ANTLR, que, sendo sincero foi um grande facilitador ao longo de todo o desenvolvimento do meu projeto.

4.2.3 Fevereiro

Com tudo definido, iniciei a fase de desenvolvimento da linguagem de programação. Comecei por criar a gramática do meu *lexer* e do meu *parser* no ANTLR. Nesta altura tinha uma ideia de que a minha gramática seria feita uma vez só, devido à minha inocência passei várias horas a olhar para a minha gramática a pensar se tinha tudo, quase como se estivesse a equipar-me para uma batalha, no início nem sequer sabia como testar se a minha gramática funcionava como queria ou não, devido a esta ansiedade e falta de conhecimento levou-me a gastar muito mais tempo nesta parte do projeto do que o que considero o ideal. Após esta fase finalmente comecei a explorar como iria fazer o código em Go propriamente dito.

4.2.4 Março a junho

Nos restantes meses, como já estava a desenvolver o projeto de maneira mais intensa a pesquisa tornou-se uma tarefa menos intensa, ou seja, em vez de fazer toda uma pesquisa para compreender um tópico, simplesmente fazia uma pequena pesquisa de maneira a entender a parte que me era útil no momento.

4.3 Desenvolvimento

Durante o mês de fevereiro comecei, efetivamente, a desenvolver o projeto, tendo feito o código inicial da CLI, foi através deste pequeno passo que ganhei confiança para experimentar e desenvolver. mas também foi quando comecei a perceber que o trabalho era maior do que estava à espera. Nesta parte do relatório irei descrever um pouco sobre como foi o desenvolvimento de cada parte que constitui a R2D2.

Assinale-se que todas as partes do projeto que envolvem a comunicação através de texto, como mensagens de erro, por exemplo, foram desenvolvidas em inglês, isto porque a maioria das linguagens não contem uma opção para trocar o idioma, bem como para incentivar utilizadores a contribuir e a utilizar a mesma, incentivando o crescimento da comunidade que foi visto como um ponto fraco no estudo de mercado.

4.3.1 Desenvolvimento da interface de linha de comandos

Esta foi a parte pela qual decidi começar a desenvolver, já que qualquer linguagem de programação necessita de uma CLI, e esta parte não seria tão diferente de qualquer aplicação na linha de comandos que tivesse feito anteriormente, aproveitei o tempo que estive a pensar e a desenvolver a minha gramática para trabalhar e aprofundar o que é que a minha interface poderia ter e, a certo ponto, deparei-me com uma biblioteca de Go chamada *Bubbletea*, que, como mencionado na secção 3.5.1.7, na sua essência é um conjunto de componentes que tornam a experiência TUI mais próxima de uma GUI (*Graphical User Interface*), achei incrível e ao vê-la eu pensei que tinha de arranjar uma forma de incluir esta biblioteca no meu projeto, e veio imediatamente o pensamento de criar um melhor comando de ajuda, eu nunca tinha gostado das opções de ajuda das linguagens de programação, que tipicamente simplesmente mostram uma lista gigante dos comandos possíveis. Tendo isto em conta decidi implementar um comando de ajuda alternativo em que em vez de serem mostrados os comandos possíveis, aparece um menu no ecrã no qual o utilizador pode navegar e pesquisar através da lista de comandos que a minha interface tem para oferecer.

4.3.2 Desenvolvimento da gramática

Esta foi a “peça” que mais dores de cabeça me deu, não particularmente pela dificuldade técnica, principalmente pela minha falta de familiaridade com *regular expressions* e pelo conhecimento que possuía no momento, que era mínimo, e por causa disso passei muito tempo desnecessário a verificar coisas que não tinha de verificar e a desenvolver coisas que não tinha de desenvolver.

Por outro lado, esta falta de conhecimento e tempo perdido de maneira desnecessária permitiram-me ter uma gramática muito melhor do que se talvez eu tivesse essa noção e que a longo prazo facilitou-me muito, já que ao longo do desenvolvimento da linguagem de programação propriamente dita não tive de fazer nenhuma alteração crítica à gramática da mesma.

4.3.3 Desenvolvimento da linguagem de programação

Esta, sendo a fase principal do desenvolvimento do projeto, é naturalmente a fase em que mais problemas me deparei, mais ansiedade me causou e mais orgulho me dá.

Inicialmente estava bastante nervoso e perdido porque não sabia exatamente o que fazer e, das coisas que sabia que tinha de fazer, como fazer. A falta de recursos no que toca ao que estava a desenvolver com as tecnologias que escolhi também não ajudou muito, mas com a prática a facilidade foi cada vez maior, para ilustrar melhor é como se fosse uma função quadrática em que o x é o tempo e o y é a facilidade (Gráfico 1).

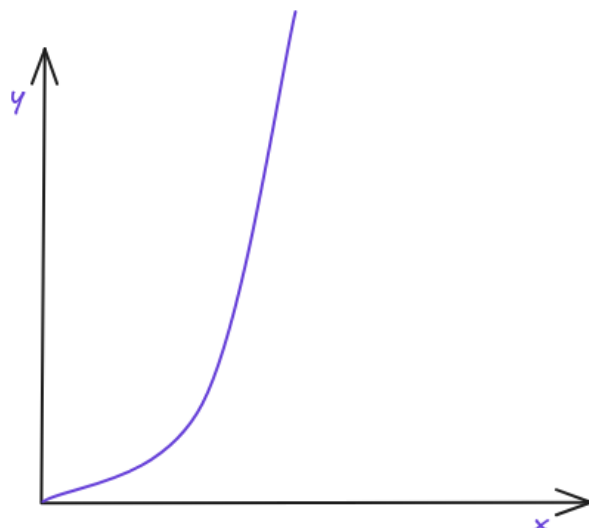


Gráfico 1 Gráfico da facilidade mediante o tempo decorrido do desenvolvimento do projeto

4.3.4 Desenvolvimento do *website*

O *website* para a minha linguagem foi a parte em que mais facilidade tive, principalmente por se tratar de um site relativamente simples, já que tudo o que ele faz é apresentar informação para o utilizador. No entanto, para contrariar a sua simplicidade, deparei-me com várias maneiras de o fazer, e *frameworks* feitos para *sites* deste género, como foi o caso do *framework Astro*, que vem com um kit destinado para sites de documentação de aplicações deste género, mas ao experimentá-las tornava-se cada vez mais evidente a ideia de que se está sempre a falar de que “menos é mais”. Por isso decidi simplificar e manter a minha *stack* consistente e desenvolver o site com a mesma linguagem que utilizei para a parte principal da PAP, para a minha sorte Go tem uma *framework* destinada para o desenvolvimento *web*, *Templ*, que apesar de não ser tão popular como as outras que tinha visto anteriormente, proporcionou-me a simplicidade e a familiaridade que eu precisava para fazer o site.

4.3.5 Desenvolvimento do Logótipo

Como qualquer linguagem de programação, a R2D2 precisava de um logótipo.

Para o desenvolvimento do logótipo recorri à minha irmã que é estudante de design gráfico e juntos discutimos ideias, e possibilidades do que o logótipo poderia ser. Após algumas ideias e ter definido o género de logótipo que queria, comecei a procurar no *Pinterest* (rede social focada na partilha de imagens) exemplos parecidos àquilo que pretendia para o meu logótipo, e após alguns exemplos, começaram a ser feitos alguns rabiscos. Após uma boa quantidade de iterações ela veio ter comigo e perguntou-me se gostava de algum, depois dar-lhe dado a minha opinião, isto é, aquilo que gostava e que não gostava tanto, e, após várias iterações, finalmente tive o prazer de lhe dizer que era do meu agrado, e pouco depois, o logótipo e outros componentes gráficos foram passados do papel para o digital. A Figura 8 é uma das últimas páginas de desenho antes da decisão final.

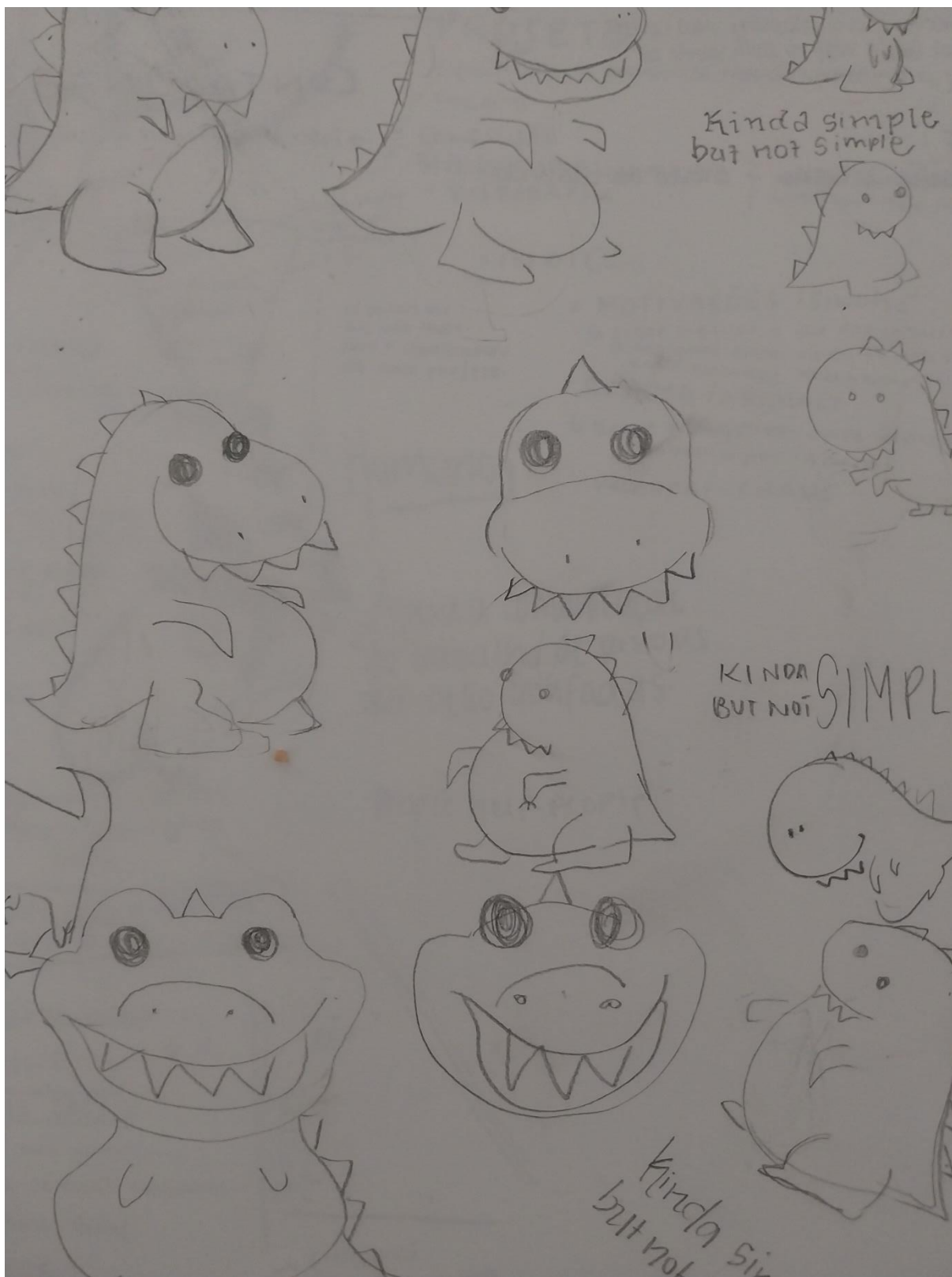


Figura 8 Página de rabiscos do logótipo

4.3.6 Testes e feedback

Gostaria de notar que grande parte do desenvolvimento se deve a testes e ao *feedback* externo.

4.3.6.1 Testes

Por volta do mês de abril, tive a oportunidade de assistir a uma palestra sobre testes, o que me deu motivação para experimentar implementar testes simples (no caso, testes unitários) no código da R2D2 e ver se, de facto, seriam uma mais valia, mesmo para um projeto em que os colaboradores são poucos ou apenas um indivíduo, como é o caso deste e, realmente, à medida que observei o projeto crescer pude perceber a importância de ter uma suite de testes, isto porque sempre que fazia uma alteração podia saber se as minhas alterações comprometeram alguma parte do projeto.

Para além dos testes mencionados acima, como é conveniente fazer em qualquer projeto, foram feitos vários testes de utilizador para verificar se o input está sempre a ser bem processado.

4.3.6.2 Feedback

Ao longo das várias fases do projeto, o *feedback* de outras pessoas demonstrou-se crucial para a concretização do mesmo. Fosse através da minha orientadora a indicar-me se eu estava adiantado ou atrasado na realização do projeto, ou outra pessoa que viu o *website* e reparou um erro de ortografia.

5 Resultado final

Neste capítulo será apresentado o projeto, especificamente, a sintaxe da linguagem, a CLI e o *website*.

5.1 Sintaxe da linguagem

A R2D2 é uma linguagem modular, em que todo o código se encontra encapsulado dentro de blocos “*module*”, e esses blocos podem conter variáveis e funções, sendo que cada módulo é transpilado para um objeto de *JavaScript*. Ao contrário da maioria das linguagens, esta linguagem destaca-se por ter uma sintaxe assertiva, ou seja, o programador é obrigado a seguir uma estrutura específica. Nos seguintes tópicos será abordada a sintaxe da linguagem.

5.1.1 Comentários

Um comentário é um conjunto de caracteres que são ignorados pela linguagem de programação, normalmente utilizados para a documentação do código. Em R2D2 existem duas formas de efetuar um comentário, através dos caracteres “//” pode ser efetuado um comentário de uma linha, caso o programador desejar um comentário que ocupe mais do que uma linha pode utilizar “/*” para dar início ao comentário e para terminá-lo “*/”. Abaixo, presentes na Figura 9, estão dois comentários, sendo que o primeiro utiliza a variante de apenas uma linha e o segundo comentário a que ocupa mais do que uma linha.

```
// Comentário de uma linha

/* Comentário
 * de várias
 * linhas
 */
```

Figura 9 Exemplo de comentários em R2D2

5.1.2 Use

No início de todos os ficheiros, é permitida a inclusão de um ficheiro R2D2 externo através da palavra-chave *use*, funcionando de maneira semelhante ao comando “include” presente na linguagem PHP (*Hypertext Preprocessor*), mas de maneira mais estrita, já que está apenas disponível para uso no início do ficheiro. Abaixo, na Figura 10, segue um exemplo da implementação do mesmo:

```
use "caminho/para/helloworld.r2d2";
```

Figura 10 Comando use em R2D2

5.1.3 Interfaces

Após o use é dada a opção de criar interfaces. Em R2D2, uma *interface* é uma definição formal das variáveis e funções que um módulo deve implementar, ou seja o utilizador define as funções e variáveis desejadas na *interface*, e depois implementa a mesma no módulo desejado. Abaixo está um exemplo para a declaração (Figura 11) e para a implementação de uma *interface* (Figura 12), respetivamente.

```
interface Hello {  
  |  
  | fn hello(name);  
  |  
}
```

Figura 11 Declaração de interface em R2D2

```
module World::Hello {  
  |  
  | fn hello(name) {  
  | |   print("Hello " + name);  
  | | }  
  |  
}
```

Figura 12 Implementação da interface "Hello" no módulo "World" em R2D2

5.1.4 Módulos

Após as *interfaces* podem ser criados os *modules* (em português, módulos). Um módulo consiste num conjunto de funções e variáveis, que, quando compilados, correspondem a um objeto de JavaScript. Note-se que para executar um código no início da com esse módulo será necessário a criação de uma função “main” que é exportada. Abaixo, na Figura 13, segue um exemplo da declaração um *module*.

```
module helloworld {
  var x;

  fn helloworld() {
    console.log("Hello World!");
  }
}
```

Figura 13 Exemplo de módulo em R2D2

5.1.5 Funções

As funções em R2D2 são um bloco de código que pode ser chamado dinamicamente e que pode conter parâmetros que são essencialmente variáveis cujos valores são definidos no momento da chamada, adicionalmente elas também podem retornar um valor através da palavra-chave “return”. Por padrão estas são apenas para uso interno no módulo (Figura 14), caso o programador queira utilizar uma função fora do seu módulo terá a opção utilizar a keyword “export” para exportar a função (Figura 15), permitindo assim a utilização noutros módulos. Adicionalmente, esta linguagem de programação introduz o conceito de pseudo-funções que podem ser declaradas através do prefixo “pseudo” (Figura 16), uma pseudo-função é uma função que apenas permite como parte da função chamadas de funções do próprio módulo, ou seja, esta não pode conter: loops, declarações de variáveis ou qualquer outra instrução que não seja uma chamada de funções do módulo interno. Abaixo está um exemplo de uma função normal, uma função exportada e uma pseudo-função, de acordo com a ordem em que foram mencionados

```
fn hello(name) {
  console.log("Hello " + name);
}
```

Figura 14 Declaração de uma função simples em R2D2

```
export fn goodbye(name) {
  console.log("Goodbye" + name);
}
```

Figura 15 Exemplo de declaração de uma função exportada em R2D2

```
pseudo fn helloworld() {
  hello("World");
  goodbye("World");
}
```

Figura 16 Exemplo de declaração de uma pseudo-função em R2D2

Para chamar uma função usa-se o nome da função seguido de um conjunto de parêntesis em que, se a função conter parâmetros, os parâmetros são colocados separados por vírgulas. Na figura abaixo (Figura 17) está representada uma chamada à função “console.log” com a cadeia de caracteres “Hello World!” como o seu único parâmetro.

```
console.log("Hello World!");
```

Figura 17 Exemplo de uma chamada de função em R2D2

5.1.6 Variáveis e constantes

Variáveis e constantes são como compartimentos para guardar valores. Há que ter em conta que a R2D2 não tem *static typing* (tipagem estática), ou seja, não se pode explicitar o tipo de uma variável. Quando compiladas as variáveis e as constantes tornam-se, respetivamente, em variáveis e constantes do *JavaScript*, tenha-se em conta que as constantes só são atribuídas um valor uma vez e esta atribuição tem de ser efetuada na sua declaração. Nas figuras abaixo está um exemplo de, respetivamente, a declaração de uma variável (Figura 18) e a declaração de uma constante (Figura 19).

```
var x;
```

Figura 18 Exemplo da declaração de uma variável em R2D2

```
const WORLD = "WORLD";
```

Figura 19 Exemplo de definição de uma constante em R2D2

5.1.7 Instruções condicionais

Nesta linguagem de programação existem duas declarações condicionais, o *if* e o *switch*.

5.1.7.1 If e else

O *if* é um comando que executa uma expressão e, caso esta expressão retorne o valor verdadeiro, equivalente ao número 1, um conjunto específico de instruções definidas pelo programador serão executadas, caso a expressão retorne falso o programador tem a opção de utilizar a palavra-chave “else” para executar outro bloco código que será executado apenas se esta condição for falsa. Na Figura 20 está a sintaxe de como um *if* pode ser implementado em R2D2.

```
if true {
  // Se o resultado for verdadeiro
} else {
  // Se o resultado for falso
}
```

Figura 20 Exemplo de implementação do *if* e do *else* em R2D2

5.1.7.2 Switch

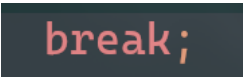
O *switch* é um comando que, de maneira semelhante ao *if*, executa uma expressão, e baseando-se na expressão executa um conjunto de comandos específicos. No entanto, no *switch* o conjunto de comandos executado não é necessariamente baseado se o resultado desta expressão é verdadeiro, mas também, através da palavra-chave “case”, basear-se num resultado definido pelo programador, este pode ser uma cadeia de caracteres, um valor booleano, entre outros. Tenha-se em conta que caso seja necessária uma saída inesperada de um case pode ser utilizada a keyword “break” para efetuar essa retirada. Abaixo, na Figura 21, está um exemplo de uma iteração possível para o *switch*.

```
switch(1) {
  case 1 {
    console.log("Hello World!");
  }
  default => console.log("Goodbye World!");
}
```

Figura 21 Exemplo de implementação do *switch* em R2D2

5.1.8 Loops

Os loops são uma instrução que repete um determinado conjunto de instruções. Em R2D2 há três maneiras de fazer loops: o *for*, o *while* e o *loop*. Vale a pena referir que cada um destes comandos permite a utilização de comandos de controlo dos mesmos: o *break* e o *continue*. O *break* (implementado na Figura 22) essencialmente força a paragem da execução do loop e o *continue* (presente na Figura 23) passa à frente o resto da execução do bloco retornando à execução para o comando inicial do loop.



```
break;
```

Figura 22 Sintaxe do *break* em R2D2

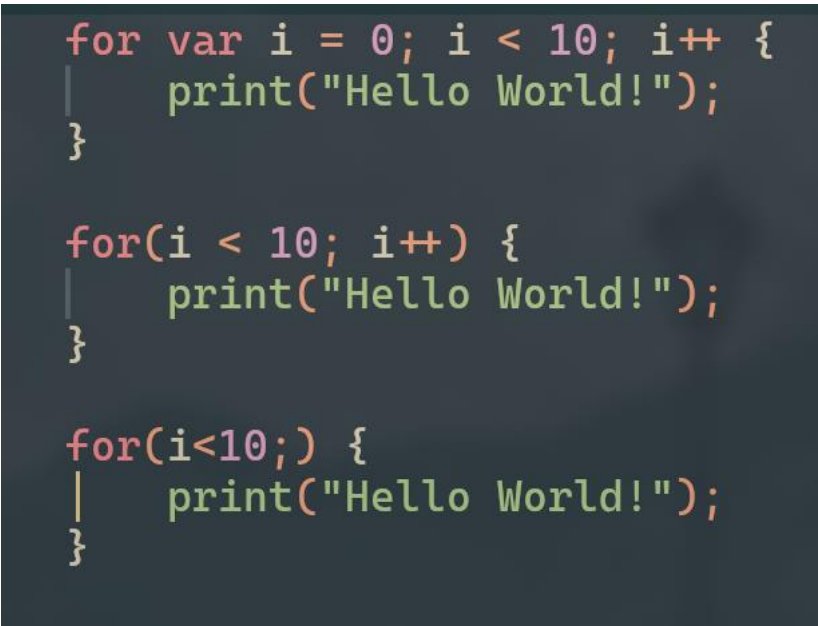


```
continue;
```

Figura 23 Sintaxe do *continue* em R2D2

5.1.9 For

O *for* tem três diretivas, em primeiro lugar, uma atribuição de variável, que será executada ao entrar no loop, em segundo lugar, uma condição que definirá se o bloco de instruções será executado ou não, e por último, outra atribuição de variável que será executada cada vez que o bloco acabar de ser executado, conste-se que todas estas diretivas são opcionais, podendo escolher fazer todas, ou até, nenhuma. Na Figura 24 estão algumas formas de como um *for* pode ser feito.



```
for var i = 0; i < 10; i++ {  
  |   print("Hello World!");  
}  
  
for(i < 10; i++) {  
  |   print("Hello World!");  
}  
  
for(i<10;) {  
  |   print("Hello World!");  
}
```

Figura 24 Exemplos de diferentes maneiras de fazer um *for*

5.1.10 While

O *while* é um loop simples que executa uma condição e apenas enquanto essa condição for verdadeira é que o bloco de código correspondido é executado. A Figura 25 é uma implementação deste comando em R2D2.

```
while true {  
  | console.log("Hello World!");  
}
```

Figura 25 Exemplo de como fazer um *while* em R2D2

5.1.11 Loop

O loop é simplesmente uma repetição infinita de um bloco de comandos. A Figura 26 mostra a sintaxe do mesmo.

```
loop {  
  | console.log("Hello World!");  
}
```

Figura 26 Exemplo da implementação do *loop*

5.2 Interface de Linha de Comandos

A linha de comandos é o local em que o programador irá executar compilar e transpilar o código R2D2. Para realizar estas tarefas a interface da R2D2 fornece os seguintes comandos: *version*, *help*, *run*, *build* e *js*.

5.2.1 Version

Através deste comando o utilizador pode verificar em que versão da R2D2 é que o seu sistema contém, assim como representado na Figura 27.

```
> r2d2  
INFO R2D2 Language – version 0.1.1
```

Figura 27 Exemplo da execução do comando *version*

5.2.2 Help

O comando *help* (ilustrado nas figuras Figura 28, Figura 29 e Figura 30), assim como indica o nome, está destinado a ajudar o programador a utilizar a CLI, ao executar o comando sem argumento é aberto um menu interativo que permite o utilizador navegar pelos comandos disponíveis (Figura 28), bem como aos argumentos dos mesmos (Figura 29). Se por alguma razão se preferir um *output* mais tradicional através de uma lista estática dos comandos, esta também está disponível adicionando o argumento “static” à frente do comando (Figura 30). Abaixo estão respetivamente, o menu inicial, a página de detalhes de um comando e a versão estática do comando.

```

R2D2 Commands

5 items

help [basic]
Shows help menu (interactive by default, use 'static' for simple output)

version [basic]
Displays the language version

build [build]
Compiles a .r2d2 file

run [build]
Executes a .r2d2 file

↑/k up • ↓/j down • / filter • q quit • ? more
/ filter • c categories • q quit • ↑/↓ navigate • enter details

```

Figura 28 Menu inicial do comando *help*

```

help
basic

Shows help menu (interactive by default, use 'static' for simple output)

Usage: r2d2 help [static]

Examples:
r2d2 help
r2d2 help static

ESC to return

```

Figura 29 Página de detalhes de um comando

```
> r2d2 help static
R2D2 Language - Help

BASIC COMMANDS:

help - Shows help menu (interactive by default, use 'static' for simple output)
Usage:  r2d2 help [static]
Examples:
    r2d2 help
    r2d2 help static

version - Displays the language version
Usage:  r2d2 version
Examples:
    r2d2 version

BUILD COMMANDS:

build - Compiles a .r2d2 file
Usage:  r2d2 build <file.r2d2>
Examples:
    r2d2 build hello.r2d2
    r2d2 build hello.r2d2 -o hi

run - Executes a .r2d2 file
Usage:  r2d2 run <file.r2d2>
Examples:
```

Figura 30 Comando *help static*

5.2.3 Run

O comando *run* (Figura 31) simplesmente executa o código de um ficheiro R2D2 no terminal. A imagem mostra a execução de um ficheiro *r2d2*.

```
> r2d2 run helloworld.r2d2
Hello, world!
SUCCESS Execution completed in 0.054 seconds
```

Figura 31 Exemplo de execução do comando *run* em R2D2

5.2.4 Build

Este comando, exemplificado nas figuras Figura 32 e Figura 33, compila um ficheiro R2D2 para um ficheiro binário. Adicionalmente pode ser adicionado o argumento `-o` para gerar um binário com um nome diferente do ficheiro R2D2.

```
> r2d2 build helloworld.r2d2
SUCCESS Created executable: helloworld

Compilation completed in 0.586 seconds
```

Figura 32 Exemplo da execução do comando *build* da CLI da R2D2

```
> r2d2 build helloworld.r2d2 -o hi
SUCCESS Created executable: hi

Compilation completed in 0.505 seconds
```

Figura 33 Exemplo da execução do comando *build* com o argumento “-o” em R2D2

5.2.5 Js

O comando *js* (Figura 34 e Figura 35) transpila o código R2D2 para um ficheiro de JavaScript. De maneira semelhante ao comando *build*, ao adicionar um argumento “-o” o utilizador pode mudar o nome do ficheiro que vai ser gerado.

```
> r2d2 js helloworld.r2d2
SUCCESS Created JavaScript file: helloworld.js
```

Figura 34 Exemplo da execução do comando *js* da linha de comandos da R2D2

```
~ via v20.19.2
> r2d2 js helloworld.r2d2 -o hi
SUCCESS Created JavaScript file: hi.js
```

Figura 35 Exemplo da execução do comando *js* da CLI da R2D2 com o argumento “-o”

5.3 Website

O *website* tem como objetivo dar uma plataforma para o programador instalar e aprender a linguagem de programação. Para esse efeito foram feitas duas páginas: a página inicial e a página da documentação.

A página inicial (Figura 36) é mais simples, já que está mais destinada para a exposição do projeto, indicando que o projeto se trata de uma linguagem de programação. No que toca a ações esta permite prosseguir para a documentação ou para o repositório *GitHub* da R2D2.

A página da documentação (Figura 37) faz uma breve introdução à linguagem, à instalação da mesma e a documentação da sintaxe da linguagem em si.

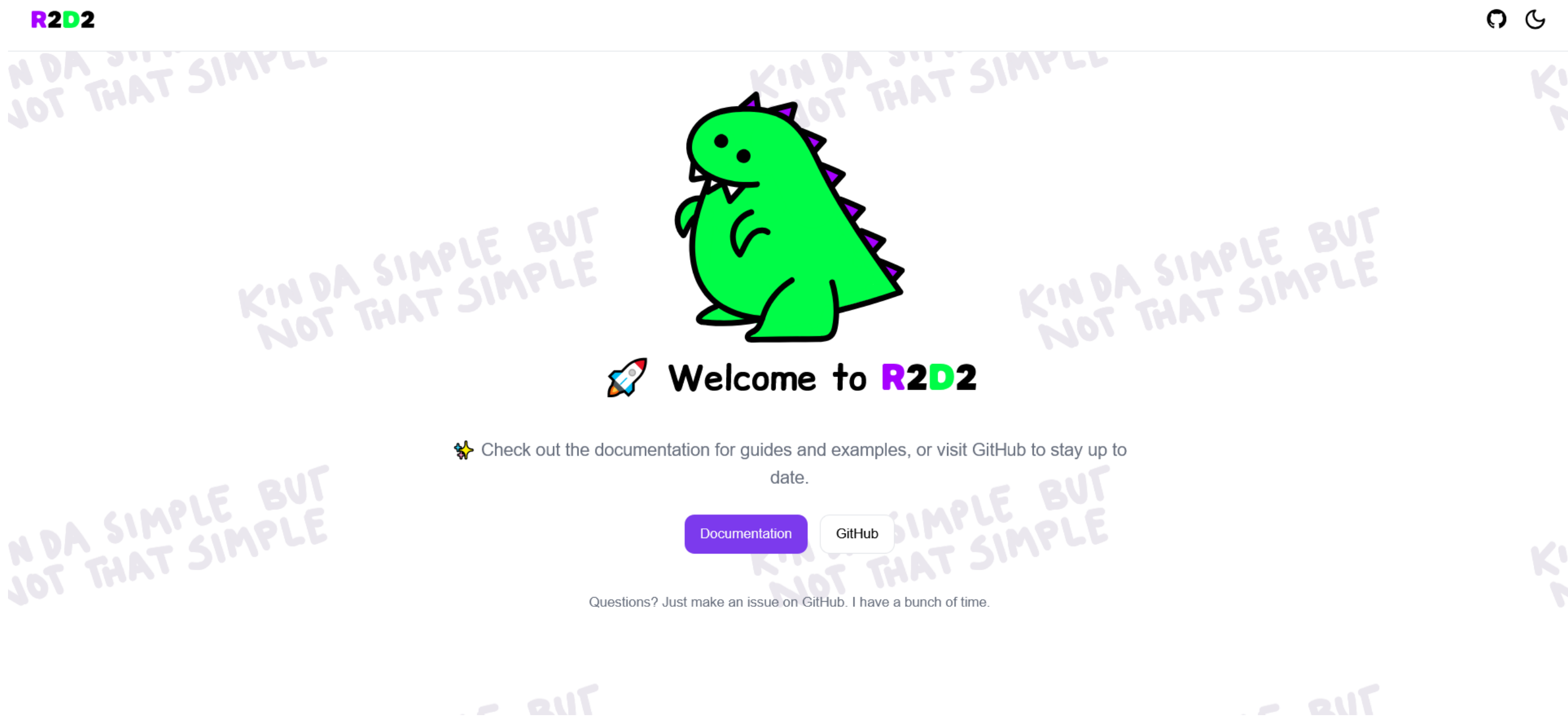


Figura 36 Página inicial

R2D2**Getting Started**[Introduction](#)[Installation](#)[Hello world](#)[Comments](#)[Variables](#)[If statements](#)[Loops](#)[Switch statements](#)[Functions](#)[Modules](#)[Interfaces](#)**Introduction**

R2D2 is a programming language designed for building modular JavaScript applications using a clear, explicit, and structured syntax.

It is written in Go and compiles to JavaScript, embracing a module-based architecture where each module can contain **variables** and **functions**. These modules are compiled into native JavaScript `objects`, making it easy to integrate with existing frontend or backend projects.

R2D2 introduces the concept of **pseudo-functions** — a special type of function that can only contain *calls to other functions*. This enforces composability and encourages building code through reusable, isolated behaviors.

The language is built with clarity in mind. It avoids hidden behavior and favors explicit patterns, making it approachable for those who want to write structured, maintainable code.

Installation

To install **R2D2**, run the following command in your terminal:

 Linux macOS Windows

```
curl -sSL https://raw.githubusercontent.com/ArturC03/r2d2-cli/main/install.sh | bash
```

After running on of the commands above, the `r2d2` binary should be available in your **\$PATH**.

You can **verify the installation** by running the following command:

```
r2d2 version
```

Figura 37 Página da documentação

6 Melhorias futuras

Sendo este um projeto de desenvolvimento de uma linguagem de programação, considera-se que nunca está verdadeiramente concluído. Existem, por isso, várias melhorias já idealizadas e planeadas para o futuro. Entre elas destacam-se: a possibilidade de executar comandos da CLI através do menu de ajuda, o desenvolvimento de uma página *web* que permita experimentar a linguagem R2D2 sem necessidade de instalação, a introdução de tipos, a deteção automática de funções *JavaScript*, a criação de uma LSP, entre outras.

Neste capítulo, serão exploradas em detalhe cada uma destas melhorias.

6.1 Executar comandos através do menu de ajuda da CLI

Neste momento o menu de ajuda é dinâmico, mas a ajuda é estática, isto é, o menu apenas fornece texto explicativo, não vai para além disso, e de maneira a poder aproveitar o menu interativo pode ser feita a adição de uma opção para executar o comando selecionado e, através de componentes disponibilizados pelo *Bubbles*, seria possível a introdução de texto e a seleção de ficheiros, tudo no comando *help* e inerente à CLI.

6.2 Página web de experimentação da linguagem

Neste momento, o utilizador tem de instalar a R2D2 num dispositivo para realmente testar a linguagem e perceber realmente como ela funciona. No entanto, linguagens como *Go* já resolveram este problema através de algo que chamam de “playground”, um *playground* é essencialmente uma página ou espaço dedicado à criação e execução de código na página. A introdução da R2D2 para uma página *web* pode ser facilmente adquirida, pois, como o site é desenvolvido utilizando um servidor em *Go*, é possível incluí-la através de um pacote de *Go*.

6.3 Tipagem estática

No início do projeto, a ideia era de fazer uma linguagem com tipos estáticos, mas à medida do desenvolvimento percebi que se me preocupasse com este assunto estaria a comprometer outros aspetos da linguagem. Por isso, o próximo passo na sintaxe da linguagem seria esta adição.

Quanto à implementação da funcionalidade. A meu ver, uma maneira viável e relativamente fácil de incluir tipos seria transpilando os tipos para notações em *JavaScript* e através de um *linter* verificar se os mesmos estão a ser respeitados.

6.4 Detecção automática de funções JavaScript

Neste momento a utilização de funções de *JavaScript* só pode ser feita de duas maneiras, criando uma função que executa um bloco puro de *JavaScript* que chama essa função ou injetando a função para a linguagem, sendo que a primeira pode ser feita no nível da programação R2D2 e a segunda apenas através da programação interna da R2D2.

Anteriormente, já foram feitas tentativas a tornar esta detecção automática disponível. No entanto, devido a falta de conhecimento técnico não foi possível tornar esta funcionalidade uma realidade. Uma solução possível para trazer esta funcionalidade seria através da execução de um script de *JavaScript* que retornasse todas as funções e os seus argumentos através de um formato de texto como, por exemplo, JSON (*JavaScript Object Notation*).

6.5 Criação de uma LSP

Hoje em dia através de um editor de código, como o *Neovim* ou o *Visual Studio Code*, é possível ver erros, de sintaxe por exemplo, sem sequer executar o código. Isto porque atualmente estão disponibilizadas LSPs (*Language Server Protocols*) que são, essencialmente, programas que entram em contacto com o editor e dão-lhe informações como erros, avisos e até dam cor ao texto de maneira a distinguir melhor de que se trata cada *token*. No entanto, ainda não foram feitos planos ou ideias de como será implementada.

6.6 Outros planos

Para além dos planos da funcionalidade do projeto, também é importante melhorar a estrutura interna do projeto, isto seria: a criação de mais testes de maneira a ter 100% de cobertura de código, a partição de certas funcionalidades em diferentes funções e outros tipos de erros ou más práticas que não tenham sido detetadas ou corrigidas anteriormente.

Conclusão

Tal como foi referido ao longo do relatório, considero que consegui atingir os objetivos que defini no início do projeto. Consegui desenvolver uma linguagem de programação funcional e, a meu ver, minimamente útil, com sintaxe própria, documentação, interface de linha de comandos e um *website* de apoio.

Ao longo deste processo, deparei-me com diversos desafios, como a complexidade da gramática, a escolha das tecnologias ou simplesmente o cansaço natural de desenvolver um projeto desta dimensão. No entanto, fui capaz de me adaptar, de aprender de forma autónoma e de ultrapassar cada dificuldade com persistência e curiosidade. Este projeto permitiu-me adquirir conhecimentos que vão muito além do que foi lecionado ao longo do curso, sobretudo nas áreas dos compiladores, da estruturação de linguagens e do desenvolvimento mais técnico e profundo de *software*. Além disso, deu-me uma nova perspetiva sobre o que sou capaz de fazer com dedicação e interesse verdadeiro.

Sinto que cresci não só como programador, mas também como pessoa. A R2D2 é o reflexo da minha criatividade, esforço e identidade — e é também uma prova pessoal de que sou capaz de começar e terminar algo que me desafia genuinamente.

Bibliografia

Adrian, A. (s.d.). *TemplUI*. Obtido em 25 de maio de 2025, de TemplUI: <https://templui.io/>

ANTLR. (s.d.). *ANTLR*. Obtido em 10 de maio de 2025, de ANTLR: antlr.org

Arch Linux. (s.d.). *Arch Linux*. Obtido em 12 de maio de 2025, de Arch Linux: <https://archlinux.org/>

Charmbracelet. (s.d.). *Bubbles*. Obtido em 12 de maio de 2025, de Github:
<https://github.com/charmbracelet/bubbles>

Charmbracelet. (s.d.). *Bubbletea*. Obtido em 12 de maio de 2025, de Github:
<https://github.com/charmbracelet/bubbletea>

Colégio de Gaia. (s.d.). *Colégio de Gaia*. Obtido em 12 de fevereiro de 2025, de Colégio de Gaia:
<https://www.colgaia.pt/>

Deno. (s.d.). *Deno*. Obtido em 10 de maio de 2025, de Deno: <https://deno.com/>

Github. (s.d.). *Github*. Obtido em 15 de maio de 2025, de Github: github.com

Go. (s.d.). *Golang*. Obtido em 10 de maio de 2025, de Golang: <https://go.dev/>

Hesketh, A. (s.d.). *Docs*. Obtido em 2 de junho de 2025, de Templ: <https://templ.guide/>

Neovim. (s.d.). *Neovim*. Obtido em 15 de maio de 2025, de Neovim: <https://neovim.io/>

Glossário

ANTLR: Ferramenta para gerar analisadores léxicos (lexer) e sintáticos (parser) a partir de gramáticas, facilitando a criação de compiladores.

API: Interface que permite a comunicação entre diferentes aplicações ou módulos, utilizada para criar endpoints na aplicação em Go.

ASTRO: Framework para desenvolvimento de *websites* rápidos e estáticos. No projeto, foi considerado para a criação da documentação, através do tema Starlight.

Bubbles: Wrapper da biblioteca Bubbletea que fornece componentes visuais aprimorados para interfaces TUI.

Bubbletea: Biblioteca em Go para construção de interfaces do tipo TUI (Text User Interface), usada na interface de linha de comandos da R2D2.

Build: Comando da CLI da R2D2 que compila um ficheiro “.r2d2” para um ficheiro binário executável.

CLI (Command Line Interface): Interface de linha de comandos utilizada para compilar, executar ou transpilar código na linguagem R2D2.

Deno: Runtime moderno para executar JavaScript e TypeScript fora do navegador, utilizado como runtime principal da linguagem R2D2.

Fork (Git): Cópia independente de um repositório que permite modificações separadas, normalmente usada para contribuir para projetos open-source.

Framework: Estrutura ou conjunto de ferramentas que facilita o desenvolvimento de aplicações, como é o caso do Templ ou do TailwindCSS.

GitHub: Plataforma de alojamento e controlo de versões baseada em Git, usada para versionar o código do projeto.

Go / Golang: Linguagem de programação usada para desenvolver o compilador e a CLI da linguagem R2D2, escolhida pela sua simplicidade e suporte a concorrência.

Help: Comando da CLI da R2D2 que apresenta ao utilizador o menu de ajuda com os comandos disponíveis.

JavaScript: Linguagem de programação para *web*. Foi o alvo final de transpiração da linguagem R2D2 devido à sua ampla utilização.

Js: Comando da CLI que transpila o código R2D2 para um ficheiro “.js” (JavaScript).

Lexer: Componente que analisa o código-fonte e divide-o em unidades chamadas tokens, utilizado na fase de análise léxica do compilador da R2D2.

Loop: Comando da linguagem R2D2 que representa um ciclo de execução infinito.

Markdown: Linguagem de marcação leve usada para criar documentos formatados, utilizada na fase de prototipagem da documentação.

Module: Bloco de código em R2D2 que encapsula variáveis e funções, transpilado para objetos JavaScript.

Parser: Componente responsável pela análise sintática do código, construindo uma estrutura interpretável com base nos tokens produzidos pelo lexer.

Pseudo-função: Tipo especial de função em R2D2 que só pode conter chamadas a outras funções do mesmo módulo, sem variáveis, ciclos ou condições.

Pull Request: Mecanismo usado no GitHub para propor alterações num repositório e solicitar a sua revisão e integração.

Run: Comando da CLI da R2D2 que executa o código diretamente no terminal.

Rust: Linguagem de programação segura e eficiente, referida no relatório como alternativa analisada à Go para o projeto.

Starlight: Tema do framework Astro, focado na criação de *websites* de documentação técnica.

Switch: Estrutura condicional da linguagem R2D2 que executa blocos de código com base em múltiplas condições (cases).

Tailwind CSS: Framework de CSS baseado em classes utilitárias, usado para estilizar o *website* do projeto.

Templ: Framework em Go para gerar HTML dinâmico através de templates, usada no desenvolvimento do *website* da R2D2.

Templ UI: Biblioteca de componentes visuais para o Templ, com estrutura flexível e fácil personalização.

Tokens: Unidades sintáticas básicas geradas na fase de análise léxica de um compilador.

TypeScript: Superset de JavaScript com tipagem estática, analisada como uma das linguagens de referência na comparação com a R2D2.

Use: Palavra-chave da linguagem R2D2 usada para importar ficheiros externos no início de um módulo.

Version: Comando da CLI da R2D2 que informa a versão atual da linguagem instalada.

Bun: Runtime moderno e rápido para JavaScript e TypeScript, considerado no projeto como alternativa ao Deno para a execução de código.

Commits: Registos de alterações feitas ao código num sistema de controlo de versões como o Git, permitindo histórico e rastreabilidade das modificações.

Interface: Em programação, define um contrato que especifica métodos que devem ser implementados por um tipo, promovendo a abstração e a modularidade.

Keyword: Palavra reservada na linguagem de programação que possui um significado especial e não pode ser usada como identificador. Exemplos: *fn*, *var*, *loop*.

Linter: Ferramenta que analisa o código-fonte à procura de erros, más práticas ou inconsistências de estilo, ajudando a manter a qualidade do código.

LSP (Language Server Protocol): Protocolo utilizado para fornecer funcionalidades como autocompletar, verificação de erros e navegação no código em editores suportados, considerado para suporte à linguagem R2D2.

Output: Saída produzida por um programa ou comando, como o resultado da execução de código em R2D2, mostrado no terminal ou gravado em ficheiros.

Plugins: Extensões que adicionam funcionalidades extra a uma aplicação, analisados como forma de expandir a linguagem R2D2 sem alterar o núcleo.

Stack: Conjunto de tecnologias e ferramentas utilizadas no desenvolvimento de um projeto, como linguagens de programação, frameworks, runtimes e bibliotecas.